

AI-Enhanced Static Analysis: Reducing False Alarms Using Large Language Models

George David Apostolidis*, Ilias Kalouptsoglou*, Miltiadis Siavvas*,
Dionysios Kehagias*, and Dimitrios Tzovaras*

* Centre for Research and Technology Hellas/Information Technologies Institute, Thessaloniki, Greece
geoapos@iti.gr, iliaskaloup@iti.gr, siavvasm@iti.gr, diok@iti.gr, dimitrios.tzovaras@iti.gr

Abstract—In modern software systems, early and accurate vulnerability detection is crucial. Traditional Static Analysis Tools (SATs) highlight potential security issues, providing fine-grained information including lines of code and vulnerability categories; however, they are hindered by a large number of false alarms. On the other hand, Artificial Intelligence (AI)-based Vulnerability Prediction (VP) has emerged as a promising alternative for vulnerability identification in software products. Nevertheless, current VP methods face important limitations, such as the granularity level of the predictions, since VP is commonly conducted at the file or function level. In this study, we examine whether the utilization of AI-based vulnerability prediction as a filtering mechanism for static analysis alerts could reduce the number of false alarms, leading to more practical Static Application Security Testing (SAST). The results of the analysis show that this approach improves the practicality of static analysis, reducing false positives, with the impact on the detection accuracy being small.

Index Terms—Vulnerability Detection, Static Analysis, Large Language Models, Actionable Alerts

I. INTRODUCTION

In modern software engineering, where the developed software products are often large, complex, and interconnected, the need to develop secure software by design has increased. In accordance with the International Standard on Software Quality ISO/IEC 25010 [1], software security is now considered a critical aspect of the Software Development Life-Cycle (SDLC). The main focus of software security is on addressing vulnerabilities that reside in source code. Commonly, software vulnerabilities are caused by errors during the code development phase, but their exploitation by external threats can lead to serious consequences both financially and in terms of data privacy, confidentiality, etc. Various techniques for software vulnerability mitigation have been proposed [2]. However, the continuous increase in security breaches and reported vulnerabilities in the National Vulnerability Database (NVD) [3] indicates that existing solutions are not sufficient.

Traditionally, the detection of vulnerabilities in the source code is performed through static analysis using tools such as Fortify, Coverity, SonarQube, CppCheck. [4], [5]. These Static Analysis Tools (SATs) detect issues via a white-box testing approach and without executing the source code, by automatically checking the source code against a list of predefined by experts rules and patterns, managing to return information on potential vulnerabilities, including their location, type, and severity [5].

However, despite its strengths, static analysis remains underutilized in practice [6], [7] due to the important challenges that it faces [8], with the most critical being the overwhelming number of False Positives (FPs) that it produces, which hinders its adoption in practice [9], [10]. In particular, the extensive alert lists produced by SATs require manual inspection by developers and code reviewers to identify the actionable alerts (i.e., alerts that correspond to actual bugs that require immediate fix). This time-consuming and labor-intensive procedure, known as triaging [6], hinders the adoption of SATs in practice. To this end, recent research attempts have focused on addressing the issue of FPs by developing methods to post-process the produced alerts to identify which ones are actionable [11]–[13].

Apart from traditional static analysis tools, another promising mechanism that has emerged relatively recently in the field of vulnerability detection is Vulnerability Prediction (VP), which aims at predicting vulnerable software components (e.g., packages, files, functions, snippets). Specifically, VP is a Machine Learning (ML)-driven mechanism, which uses software attributes, such as software metrics [14], [15] or textual features [15]–[18] to classify software components as vulnerable or clean. Contrary to traditional static analysis that is based on the identification of violations of specific rules and best practices, VP is able to detect more complex vulnerability patterns, due to the utilization of advanced AI algorithms. More specifically, with the emergence of the Transformer architecture [19] and the Large Language Models (LLMs), enhanced VP solutions have been proposed [20], [21].

However, the vast majority of the proposed Vulnerability Prediction Models (VPMs) perform predictions at class- or function-level of granularity [2], [20], and are not able to provide specific information about the actual location and type of the detected vulnerability. Although there have been some studies recently that utilize explainable AI (XAI) techniques to trace the vulnerability patterns detected by the VPMs at the line level [22]–[24], they exhibit important drawbacks, the most significant being the inclusion of spurious features (i.e., code constructs that are irrelevant to the existence of vulnerabilities) thereby misleading the detection process [25], [26].

Based on the above analysis, it is clear that both SATs and VPMs are useful for detecting vulnerabilities that reside in the

source code of software applications, with each one providing its own benefits and facing its own challenges. Hence, instead of relying solely on one of them for vulnerability detection (e.g., traditionally SATs and recently VPMs), an interesting direction would be to examine whether the utilization of AI-based vulnerability prediction models could help improve the practicality of traditional static analysis, by reducing the volume of false alarms. This can be expressed more formally in the following research question:

- **RQ:** Can we leverage AI-based vulnerability prediction models for improving the practicality of static analysis?

To this end, in the current study we investigate whether the utilization of AI-based VPMs could reduce the number of false positives produced by static analysis, thereby improving its practicality, and analyzing potential consequences for overall detection accuracy. More specifically, we use a popular static code analyzer as the primary SAT to identify potential security issues and an LLM-based VPM to predict vulnerable functions. Only those alerts associated with functions marked as vulnerable by the LLM-based VPMs are retained and considered actionable, with the rest being considered non-actionable. An analysis of the impact of this filtering process on the practicality and predictive performance of the selected static analysis tool is performed, based on two well-known and widely-used vulnerability datasets containing real-world vulnerabilities.

The structure of this study is as follows. Section II provides an overview of the related literature. Subsequently, Section III describes the proposed methodology, and Section IV presents the evaluation results of our analysis. Finally, in Section VI the study is concluded, and directions for future work are proposed.

II. RELATED WORK

This section presents the related work on vulnerability detection techniques used to identify vulnerabilities in source code. We discuss the existing static code analysis techniques and the background on the more recent concept of vulnerability prediction.

A. Static Code Analysis

Several research initiatives that aim to enhance software quality and security based on static analysis have been proposed over the years [5], [27]. Static analysis tools play a vital role in evaluating source code and detecting potential vulnerabilities without requiring code execution. Their ability to identify issues early in the SDLC assists engineers in coding defensively. This is particularly beneficial in agile development, in cases where teams prioritize continuous delivery [28].

Well-known SATs in both industry and research are Fortify, Checkmarx, Veracode, and Coverity among others [4], [29]. Though used extensively in both academia and business, SATs have well-documented limits that compromise their efficacy. Their tendency to produce false positives, which are warnings for issues that do not really exist or do not pose an actual

security threat, is an important disadvantage that may overwhelm developers, reduce trust, and increase inspection effort [6], [30]. Furthermore, in attempts to minimize false positives, many tools adopt unsound analysis techniques that trade off precision for performance, often increasing false negatives (i.e., missing real vulnerabilities) [30]. Researchers emphasize that such tools often fail to provide efficient triage and prioritization of alerts, reducing their impact in real-world scenarios [31].

To address this challenge, researchers have explored various techniques to identify actionable alerts, i.e., alerts that correspond to actual bugs (including vulnerabilities) that require an immediate fix [11]. Various Actionable Alert Identification Techniques (AAITs) have been examined over the years, being based either on filtering out non-actionable alerts [12], or prioritizing the produced alerts based on their likelihood to be actionable [13]. Among the two types, the former is more appealing, as it leads to a significant reduction in the number of alerts reported by the SAT to the developers, and, in turn, in the time and effort required for checking and fixing the alerts.

However, the main challenge that existing AAITs face is their negative impact on the SAT's accuracy. More specifically, in their attempt to eliminate false positives, they inevitably introduce false negatives (i.e., they omit alerts that correspond to actual issues). Despite the advances in the field and the sophisticated techniques that have been employed, no AAITs exist that are able to eliminate false alarms without omitting actual issues. LLMs could potentially be a promising solution to alleviate this issue.

B. Vulnerability Prediction

As highlighted in a recent systematic mapping study [2], significant progress has been made in the field of vulnerability prediction with the advancement of Machine Learning (ML) and Deep Learning (DL) techniques. Both software metrics (e.g., complexity, cohesion) [32], [33] and text mining techniques [16], [17], [34] have been thoroughly examined in the VP field, with the latter producing the best results [14], [15]. Moreover, text-rich graphs have been employed for better code representation, using Abstract Syntax Trees, Program Dependence Graphs, and Code Property Graphs [35], providing a promising direction in VP [36], [37].

Following the advances in the field of Natural Language Processing (NLP) with the emergence of the Transformer architecture [19], Large Language Models (LLMs) have emerged as a promising approach for VP. Their ability to understand natural and programming languages, code semantics, and contextual patterns enables more accurate prediction of vulnerabilities [20]. A comparison of word2vec, fastText, and pre-trained Bidirectional Encoder Representations from Transformers (BERT) [38] embeddings was conducted by Bagheri et al. [39], while Kim et al. [40] proposed VulDeBERT, a BERT model fine-tuned for VP. In addition, a comparative analysis of various DL-based VPMs, including several LLMs, was performed by Steenhoek et al. [21], demonstrating the promising capacity of

LLMs in VP. Kalouptoglou et al. [41] also showed, among others, that LLMs surpass previous VPMs, including both text mining-based and graph-based approaches.

However, most VPMs make predictions at file- or function-level of granularity [2]. Recent attempts to highlight specific lines that are vulnerable leverage XAI techniques, such as Self-Attention [42], in order to identify the tokens and therefore the lines of code that contribute the most to a function-level decision of the VPM [22], [24]. Nevertheless, explanatory studies have discussed the limitations of this approach, since VPMs often make correct predictions by learning spurious data associations [25], [26]. Furthermore, creating such line-level models requires large databases with fine-grained labels, making the extension of such models across different programming languages a particularly difficult process.

Therefore, contrary to SATs which are able to pinpoint the exact lines of code where the vulnerability resides, AI-based VPMs, despite their observed higher accuracy in detecting vulnerability patterns (especially complex ones) than SATs, currently fail to provide information on the exact location and type of the detected vulnerability, which is highly useful and necessary for their correction by the development team. This greatly hinders their adoption in practice, as developers and engineers need to know the exact location and type of the detected vulnerability in order to be able to fix it, rendering the traditional static analysis still a more reliable approach.

C. Beyond State-of-the-Art

The above analysis illustrates the need for combinatorial strategies to enhance existing SAST approaches. Although initial efforts have been made to reduce the number of static analysis alerts to focus on those that are actionable, it is clear that there is a need for integrated approaches that can combine intelligent alert filtering with the fine-grained information provided by static analysis. LLMs can be used to prioritize alerts that are more likely to be actionable, thereby greatly reducing the number of false alarms and, therefore, improve the practicality of static analysis outputs [43]. To this end, this paper uses static code analysis as its core method for detecting potential vulnerabilities and employs LLMs to prioritize the detected issues. In particular, by removing alerts from functions not considered vulnerable by the LLM-based VPMs we can isolate alerts that are more likely to correspond to real vulnerabilities. Thus, this method provides an important reduction in the effort required by developers and code reviewers to find vulnerabilities among the large lists of static analysis alerts.

III. STUDY DESIGN

This section presents the SAST methodology that is examined in this study. Initially, the vulnerability-related dataset that is used to conduct our analysis is described. We then describe the approach and evaluation procedure that we follow. Figure 1 provides an overview of the approach. Specifically, the source code of the functions of the vulnerability-related dataset is

analyzed using both static analysis and VP, and the results are merged in order to retain those alerts found in the predicted-as-vulnerable functions. Those alerts, which include fine-grained information (i.e., lines and categories of vulnerabilities), are provided to a security expert or developer for review.

A. Dataset

Regarding the datasets we used in this work, we chose two well-accepted vulnerability datasets that contain real-world vulnerabilities, namely the Big-Vul [44] and the ReVeal [37] datasets. They comprise C/C++ source code samples (i.e., functions) collected from publicly available GitHub repositories by linking commits to known vulnerabilities in the CVE Database. In order to conduct our experiments, parts of these datasets were used for training and validating the LLM-based VPM, whereas the test sets of the datasets were used as benchmarks to assess the benefits achieved by the examined approach (i.e., augmenting static analysis with information retrieved from VPMs), as opposed to utilizing solely traditional static analysis without any kind of alert filtering.

B. Methodology

This section presents the methodology adopted in this study. We combined static code analysis with AI-based Vulnerability Prediction to enhance the identification of actionable static analysis alerts and, therefore, the detection of vulnerabilities. The entire process is illustrated in Figure 1 and comprises several key steps, including data preparation, static analysis, vulnerability prediction, merging the results, and comparative evaluation between the proposed methodology and the standalone static analysis solution.

First, we built a tool that employs the open-source CppCheck SAT to identify security-specific static analysis alerts in the functions of the datasets. In particular, we utilized the CppCheck plugin of the SonarQube platform in order to leverage the SonarQube API [45] for accessing the results of the analysis. The toolchain was implemented and configured in one of our prior studies [46].

In parallel, the same functions were fed to our VPM. First, they were tokenized and then given as input to the VPM to classify them as vulnerable or non-vulnerable. For the purposes of the current analysis, we employed a model developed in our previous work [41] by fine-tuning the CodeBERT LLM [47].

After both static analysis and VP were completed, their findings were combined. Specifically, we filtered out the functions that were not among those predicted as vulnerable by the VPM. Hence, we kept only those alerts reported by CppCheck that are included in predicted-as-vulnerable functions. This post-processed list of static analysis alerts was then given to the security reviewer to inspect and repair them.

Concisely, in the proposed method, VP predicts which functions are vulnerable, while the SAT identifies the lines and the categories of the security issues inside those functions. In this way, the proposed methodology reduces the inspection

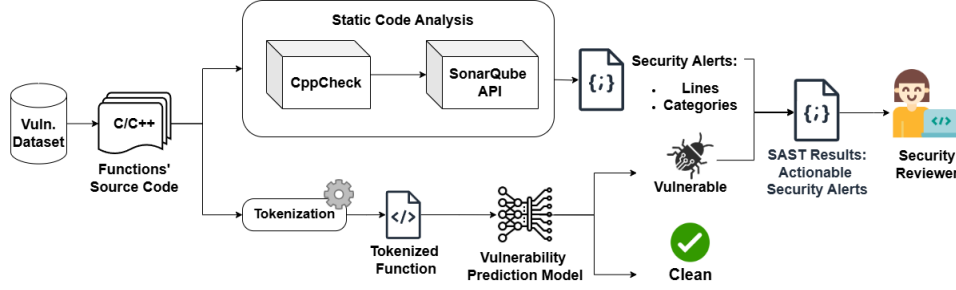


Fig. 1. High-level overview of the methodology that combines static analysis with vulnerability prediction.

effort required to manually identify the actual vulnerabilities among all the identified issues, as a non-trivial number of alerts (i.e., those that belong to benign functions) are filtered out and not shown to the reviewer/developer immediately. Due to the inevitable Type I and Type II errors of the utilized models and techniques, an evaluation of both the detection efficacy and practicality of the proposed approach is conducted and presented in the next section (Section IV), in order to assess the benefits and shortcomings of the approach with respect to accuracy, precision, and practicality.

C. Evaluation Schema

To evaluate the proposed methodology, an evaluation scheme that compares traditional static analysis with our AI-enhanced SAST approach was designed. The focus of the evaluation is on the capacity of the proposed method to reduce false positives generated by the selected static analysis tool and, in turn, the effort required for reviewing the static analysis findings without generating (at least too many) false negatives. This is important because it is known in the literature that AAITs, especially the classification-based techniques, inevitably introduce false negatives due to the filtering process that they employ (i.e., filtering out alerts that are considered non-actionable) [11]–[13]. Therefore, a quantification of this impact is important in order to evaluate whether the proposed AAIT is viable or unrealistic, i.e., whether the benefit in inspection effort reduction that is obtained by the model outweighs the omission of a small number of actionable alerts or the observed omissions are so many that any benefit in effort reduction is deemed worthless.

We computed the Precision, Recall, F_1 -score, and False Positive Rate (FPR) metrics. Given our focus on reducing FPs while identifying as many vulnerabilities as possible, the F_1 -score is utilized as the primary evaluation metric since it balances Precision and Recall through their harmonic mean. F_1 -score is defined as:

$$F_1\text{-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1)$$

Moreover, we consider the Inspection Ratio ($\mathcal{I}$), which indicates the proportion of functions that need to be reviewed

in order to identify all the actual vulnerabilities in the dataset. Lower values of $\mathcal{I}$ indicate reduced manual inspection overhead, making it especially relevant for real-world triage and auditing settings. This metric, which has been frequently used in the related literature [14], [48], captures the practical effort of manual inspection required and is defined as:

$$\mathcal{I} = \frac{TP + FP}{TP + FP + TN + FN} \quad (2)$$

This equation is used to as a measure of practicality.

IV. RESULTS AND DISCUSSION

In this section, we evaluate whether the adoption of vulnerability prediction can improve the practicality of static analysis, examining the impact on the detection accuracy.

More specifically, the purpose of this evaluation is to investigate whether the use of LLMs can reduce the FPs of static analysis. To achieve this we examine the detection accuracy and inspection ratio of static analysis with and without the adoption of VPMs. As already mentioned, the analysis was based on two well-accepted vulnerability datasets, the Big-Vul and the ReVeal.

In the Big-Vul codebase, static analysis achieved a Precision of 5.88% and a Recall of 46.90%, resulting in a low F_1 -score of 10.46. The approach required the inspection of more than 50% of the dataset (Inspection Ratio = 50.49%) and produced a large number of false positives (FPR = 47.07%).

The AI-enhanced SAST methodology greatly improved Precision to 96.20%, essentially eliminating the majority of the false positives produced by the conventional static analysis. On the other hand, Recall dropped to 40.66%, while the overall F_1 -score increased to 57.16%. In addition, this methodology achieved a FPR of only 0.10%, demonstrating how effectively it filters out non-actionable alerts. Moreover, the Inspection Ratio also decreased to 2.44%, showing that this approach is more practical in real-world settings, managing to reduce the inspection effort required to find the actionable alerts.

As we can see, the results confirm the trade-off between Precision and Recall when combining the static analysis with

TABLE I
COMPARISON OF VULNERABILITY DETECTION METHODS ON BIG-VUL AND REVEAL DATASETS

Dataset	Method	Precision	Recall	F ₁ -score	FPR	Inspection Ratio
Big-Vul	Static Analysis Only	5.88%	46.90%	10.46	47.07%	50.49%
	Proposed SAST	96.20%	40.66%	57.16	0.10%	2.44%
ReVeal	Static Analysis Only	8.71%	6.88%	7.69	7.89%	15.41%
	Proposed SAST	47.67%	5.93%	10.56	0.71%	1.34%

the LLM. High false positive rates are often produced by the traditional SATs. When the VP is combined with static analysis, the Precision of the alerts improves greatly (Precision increased by approximately 90%) at the cost of reduced Recall (Recall decreased by approximately 6%). However, the reduction in Recall is much smaller compared to the improvement in Precision and Inspection Ratio, which may be considered preferable.

With respect to the ReVeal dataset, traditional static analysis achieved a Precision of 8.71%, Recall of 6.88%, and F₁-score of 7.69. The FPR was 7.89%, and the Inspection Ratio was 15.41%. On the contrary, the combined approach was able to lower the FPR to 0.71%, confirming our method's filtering capacity. Although Recall slightly dropped to 5.93%, resulting in an F₁-score of 10.56, the Inspection Ratio was by far lower (i.e., equal to 1.34%). The similar trend seen in the Big-Vul benchmark is confirmed by the result that the combinatorial method, although it does lead to a slight drop in Recall, improves Precision and reduces the inspection effort.

Overall, the study shows that even while traditional static analysis has slightly higher Recall, it has much lower Precision and a significant number of false positives. With low FPR and Inspection Ratios in both codebases, the proposed SAST approach greatly increases Precision while lowering noise. By lowering the Inspection Ratio to less than 3% in both cases, the approach demonstrates a high practical value. Concisely, we can argue that it is feasible to create a hybrid approach for accurate and practical vulnerability detection through the combination of static analysis with VP; however, this comes at the cost of a decrease in Recall, a known inherent issue of all AAITs in the literature, but this decrease is observed to be relatively small in our examined case. In addition, this drop in Recall may be considered acceptable in some domains.

V. THREATS TO VALIDITY

This section provides a remark on threats to the validity of our study. Concerning construct validity, potential threats arise from the chosen evaluation metrics. We employ widely adopted metrics such as Precision, Recall, F₁-score, and FPR, as well as an effort-aware metric (i.e., Inspection Ratio), mitigating this risk through alignment with established literature [2], [14].

Regarding internal validity, a key threat relates to potential errors or biases in the implementation of our approach. We carefully implemented a toolchain integrating CppCheck through SonarQube and a fine-tuned LLM-based VPM, but

implementation errors might still exist. To mitigate this risk, we thoroughly reviewed our code and have also made the developed code publicly available to facilitate future replication and validation of our findings [49].

Finally, external validity threats concern the generalization of the findings, since we only used C/C++ data and the static analyzer CppCheck. However, the datasets used, which are Big-Vul [44] and ReVeal [37], are both widely referenced in vulnerability research [2], [22], [23], [41]. In addition, future research aims to explore additional languages, datasets, and analyzers to strengthen generalization.

VI. CONCLUSION AND FUTURE WORK

In this study, we examined whether the practicality of static analysis could be improved through the adoption of AI-based vulnerability prediction. More specifically, we used the findings of an LLM-based VPM to filter out static analysis alerts that are not actionable (i.e., that do not correspond to actual vulnerabilities). The evaluation results were promising, demonstrating a significant decrease in the volume of the reported false alarms, with a relatively small omission of actionable alerts. This suggests that using LLM-based VPMs may be a viable solution for improving the practicality of static analysis, especially in contexts where a slight decrease in detection accuracy can be considered acceptable.

Several directions for future work can be identified. After showing in this study that the use of LLMs as function-level vulnerability predictors can reduce the false alarms of traditional static analysis, next steps include investigating whether this approach surpasses existing AAITs by conducting a large-scale comparative study. Specific focus will be given on the drop in the Recall, in order to identify which technique achieves a better trade-off between Recall and Precision. We also aim at exploring whether the filtered-out alerts are of high or low severity. In addition, we are planning to analyze the impact of the studied approach by conducting a user study in order to receive feedback from developers and validate the practical usability of the filtered alerts, considering also the computational cost. We are also interested in repeating the current study using different SATs and for different programming languages.

ACKNOWLEDGMENT

Work reported in this paper has received funding from the European Union's Horizon Europe Research and Innovation Program through the DOSS project, Grant Number 101120270.

REFERENCES

- [1] ISO/IEC, *ISO/IEC 25010*, 2011.
- [2] I. Kaloutsoglou, M. Siavvas, A. Ampatzoglou, D. Kehagias, and A. Chatzigeorgiou, "Software vulnerability prediction: A systematic mapping study," *Information and Software Technology*, 2023.
- [3] "Published CVE Records," <https://www.cve.org/about/Metrics>, 2024.
- [4] J. Wits, "The top 8 static code analysis solutions," <https://expertinsights.com/insights/the-top-static-code-analysis>, 2024.
- [5] M. Siavvas, E. Gelenbe, D. Kehagias, and D. Tzovaras, "Static analysis-based approaches for secure software development," in *Security in Computer and Information Sciences: First International ISCIS Security Workshop 2018*. Springer International Publishing, 2018, pp. 142–157.
- [6] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *2013 International Conference on Software Engineering*. IEEE, 2013.
- [7] C. Vassallo, S. Panichella, F. Palomba, S. Proksch, H. C. Gall, and A. Zaidman, "How developers engage with static analysis tools in different contexts," *Empirical Software Engineering*, 2020.
- [8] T.-T. Nguyen and H. D. Vo, "Context-based statement-level vulnerability localization," *Information and Software Technology*, 2024.
- [9] B. Aloraini, M. Nagappan, D. M. German, S. Hayashi, and Y. Higo, "An empirical study of security warnings from static application security testing tools," *J. Syst. Softw.*, 2019.
- [10] M. Siavvas, I. Kaloutsoglou, D. Tsoukalas, and D. Kehagias, "A self-adaptive approach for assessing the criticality of security-related static analysis alerts," in *Computational Science and Its Applications*, 2021.
- [11] S. Heckman and L. Williams, "A systematic literature review of actionable alert identification techniques for automated static code analysis," *Information and Software Technology*, vol. 53, no. 4, pp. 363–387, 2011.
- [12] B. Johnson, S. Heckman, and L. Williams, "A comparative evaluation of static analysis actionable alert identification techniques," in *Proceedings of the 2013 International Conference on Software Engineering*, 2013.
- [13] T. Muske and A. Serebrenik, "Survey of approaches for handling static analysis alarms," in *2016 IEEE 16th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2016.
- [14] J. Walden, J. Stuckman, and R. Scandariato, "Predicting vulnerable components: Software metrics vs text mining," in *2014 IEEE 25th international symposium on software reliability engineering*, 2014.
- [15] I. Kaloutsoglou, M. Siavvas, D. Kehagias, A. Ampatzoglou, and A. Chatzigeorgiou, "Examining the capacity of text mining and software metrics in vulnerability prediction," *Entropy*, vol. 24, no. 6, p. 839, 2022.
- [16] R. Scandariato, J. Walden, A. Hovsepyan, and W. Joosen, "Predicting vulnerable software components via text mining," *IEEE Trans. Softw. Eng.*, vol. 40, no. 10, pp. 993–1006, 2014.
- [17] I. Kaloutsoglou, M. Siavvas, D. Kehagias, A. Chatzigeorgiou, and A. Ampatzoglou, "An empirical evaluation of the usefulness of word embedding techniques in deep learning-based vulnerability prediction," in *International ISCIS Security Workshop*. Springer, 2022, pp. 23–37.
- [18] I. Kaloutsoglou, M. Siavvas, A. Ampatzoglou, D. Kehagias, and A. Chatzigeorgiou, "Vulnerability prediction using pre-trained models: An empirical evaluation," in *2024 32nd International MASCOTS Conference*. IEEE, pp. 1–6.
- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in neural information processing systems*, 2017, pp. 5998–6008.
- [20] M. Siavvas, I. Kaloutsoglou, E. Gelenbe, D. Kehagias, and D. Tzovaras, "Transforming the field of vulnerability prediction: Are large language models the key?" in *2024 32nd International MASCOTS Conference*. IEEE, pp. 1–6.
- [21] B. Steenhoeck, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," 2022.
- [22] M. Fu and C. Tantithamthavorn, "Linevul: A transformer-based line-level vulnerability prediction," in *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*, 2022, pp. 608–620.
- [23] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proceedings of the 29th ACM Joint Meeting on ESEC-FSE*, 2021, pp. 292–303.
- [24] C. Pornprasit and C. K. Tantithamthavorn, "Deeplinedp: Towards a deep learning approach for line-level defect prediction," *IEEE Trans. Softw. Eng.*, vol. 49, no. 1, pp. 84–98, 2022.
- [25] A. Sotgiu, M. Pintor, and B. Biggio, "Explainability-based debugging of machine learning for vulnerability discovery," in *17th International Conference on Availability, Reliability and Security*, 2022.
- [26] B. Cheng, S. Zhao, K. Wang, M. Wang, G. Bai, R. Feng, Y. Guo, L. Ma, and H. Wang, "Beyond fidelity: Explaining vulnerability localization of learning-based detectors," *ACM Trans. Softw. Eng. Methodol.*, 2024.
- [27] M. G. Siavvas, K. C. Chatzidimitriou, and A. L. Symeonidis, "Qatch-an adaptive framework for software product quality assessment," *Expert Systems with Applications*, vol. 86, pp. 350–366, 2017.
- [28] T. Oyetoyan, D. S. Cruzes, M. Felderer, and D. Pfahl, "Developers' perceptions of security challenges in agile development," in *Software Engineering and Software Management*. Springer, 2018, vol. 304.
- [29] P. Ferrara, A. K. Mandal, A. Cortesi, and F. Spoto, "Static analysis for discovering iot vulnerabilities," *International Journal on Software Tools for Technology Transfer*, vol. 23, no. 1, pp. 71–88, 2021.
- [30] P. Emanuelsson and U. Nilsson, "A comparative study of industrial static analysis tools," *Electronic Notes in Theoretical Computer Science*, 2008.
- [31] S. Wagner and M. Fischer, "Evaluation of static analysis alert ranking through actionable warnings," *Empirical Software Engineering*, 2020.
- [32] Y. Shin and L. Williams, "Is complexity really the enemy of software security?" in *4th ACM workshop on Quality of protection*, 2008, pp. 47–50.
- [33] —, "An empirical model to predict security vulnerabilities using code complexity metrics," in *Second ACM-IEEE international symposium on international software engineering and measurement*, 2008.
- [34] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "Vuldeepecker: A deep learning-based system for vulnerability detection," *arXiv preprint arXiv:1801.01681*, 2018.
- [35] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, pp. 590–604.
- [36] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *arXiv preprint arXiv:1909.03496*, 2019.
- [37] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet," *IEEE Trans. Softw. Eng.*, 2021.
- [38] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [39] A. Bagheri and P. Hegedűs, "A comparison of different source code representation methods for vulnerability prediction in python," in *Quality of Information and Communications Technology*. Cham: Springer International Publishing, 2021, pp. 267–281.
- [40] S. Kim, J. Choi, M. E. Ahmed, S. Nepal, and H. Kim, "Vuldebert: A vulnerability detection system using bert," in *IEEE International Symposium on Software Reliability Engineering Workshops*, 2022.
- [41] I. Kaloutsoglou, M. Siavvas, A. Ampatzoglou, D. Kehagias, and A. Chatzigeorgiou, "Transfer learning for software vulnerability prediction using transformer models," *J. Syst. Softw.*, vol. 227, p. 112448, 2025.
- [42] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv:1409.0473*, 2014.
- [43] M. Bedoya, S. Palacios, D. Díaz-López, E. Laverde, and P. Nespoli, "Enhancing devsecops practice with large language models and security chaos engineering," *International Journal of Information Security*, 2024.
- [44] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "Ac/c++ code vulnerability dataset with code changes and cve summaries," in *Proceedings of the 17th international conference on mining software repositories*, 2020.
- [45] SonarSource, "Code quality, security and static analysis tool with sonarqube," <https://www.sonarsource.com/products/sonarqube/>, 2025.
- [46] M. Siavvas, D. Tsoukalas, M. Jankovic, D. Kehagias, and D. Tzovaras, "Technical debt as an indicator of software security risk: a machine learning approach for software development enterprises," *Enterprise Information Systems*, vol. 16, no. 5, p. 1824017, 2022.
- [47] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.
- [48] Y. Shin and L. Williams, "Can traditional fault prediction models be used for vulnerability prediction?" *Empirical Software Engineering*, vol. 18, pp. 25–59, 2013.
- [49] CERTH AI & SoftEng Group, "LIFESEC 2025 Experiments," <https://github.com/certh-ai-and-softeng-group/lifesecc-experiments>, 2025.